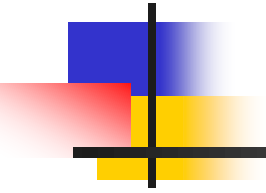
Fundamental of Software Engineering (SENG 1061)



Chapter 02

Software Process

Topics Covered

- Software Process
- Characteristics of a software process
- Software Development Process
- Software life cycle and process model
- Process Assessment Model
- Software Process Metrics

Software Process.....?

- The software process defines the **way in which software development is organized**, **managed**, **measured**, **supported** and **improved**. (independently of the type of support technology exploited in the development)



- All **organizations involved in software development follow a process of some kind**, implicit or explicit, reproducible, instrumented or adaptable .

Conti…

- **A structured set of activities required to develop a software system**
- Organizing a structured set of activities to develop software systems.
- Many different software processes involve the following activities:
  - **Specification** – defining what the system should do;
  - **Design and implementation** – defining the organization of the system and implementing the system;
  - **Validation** – checking that it does what the customer wants;
  - **Evolution** – changing the system in response to changing customer needs.

SDLC

- Main phases of software development process are:
- **Requirements Analysis** (answers “WHAT?”)
  - Specifying what the software must do
- **Design** (answers “HOW?”)
  - Specifying different parts of the s/w, and how they will fit together
- **Implementation** (a.k.a. “CODING”)
  - Writing the code
- **Testing**
  - Validating the product against requirements.
- **Maintenance** (Fixing or Enhancing)
  - » Fixing the defects and adding new capabilities

The Software Process Model

- A process model (also called software life cycle model) is a descriptive and diagrammatic representation of the software life cycle.
- A **process model** is an *abstract representation of a process*. It presents a description of a process from some particular perspective
- A **process model** maps the different activities performed on a software product from its inception to retirement.
- Thus, **no matter which process model is followed**, the basic activities are included in all life cycle models though the activities may be carried out in different orders in different life cycle models

Need for Software Process Model

- The software development team must have a clear understanding about **when and what to do**
- Without using of a particular process model
  - the development of a software product would not be in a systematic and disciplined manner.
  - it becomes difficult for software project managers to monitor the progress of the project.
- A **software process model defines entry and exit criteria for every phase**.
  - A phase can start only if its phase-entry criteria have been satisfied.

Different Software Process models

- Many software process models have been proposed so far. Each of them has some advantages as well as some disadvantages. A few important and commonly used Software Process models are as follows:
  1. Waterfall Model
  2. V-Shaped SDLC Model
  3. Structured Evolutionary Prototyping Model
  4. Iterative Waterfall Model RAD
  5. Incremental Model
  6. Spiral Model
  7. The (Rational) Unified Process
  8. Agile Model
- **The development team must identify a suitable Process model for the particular project and then adhere to it.**

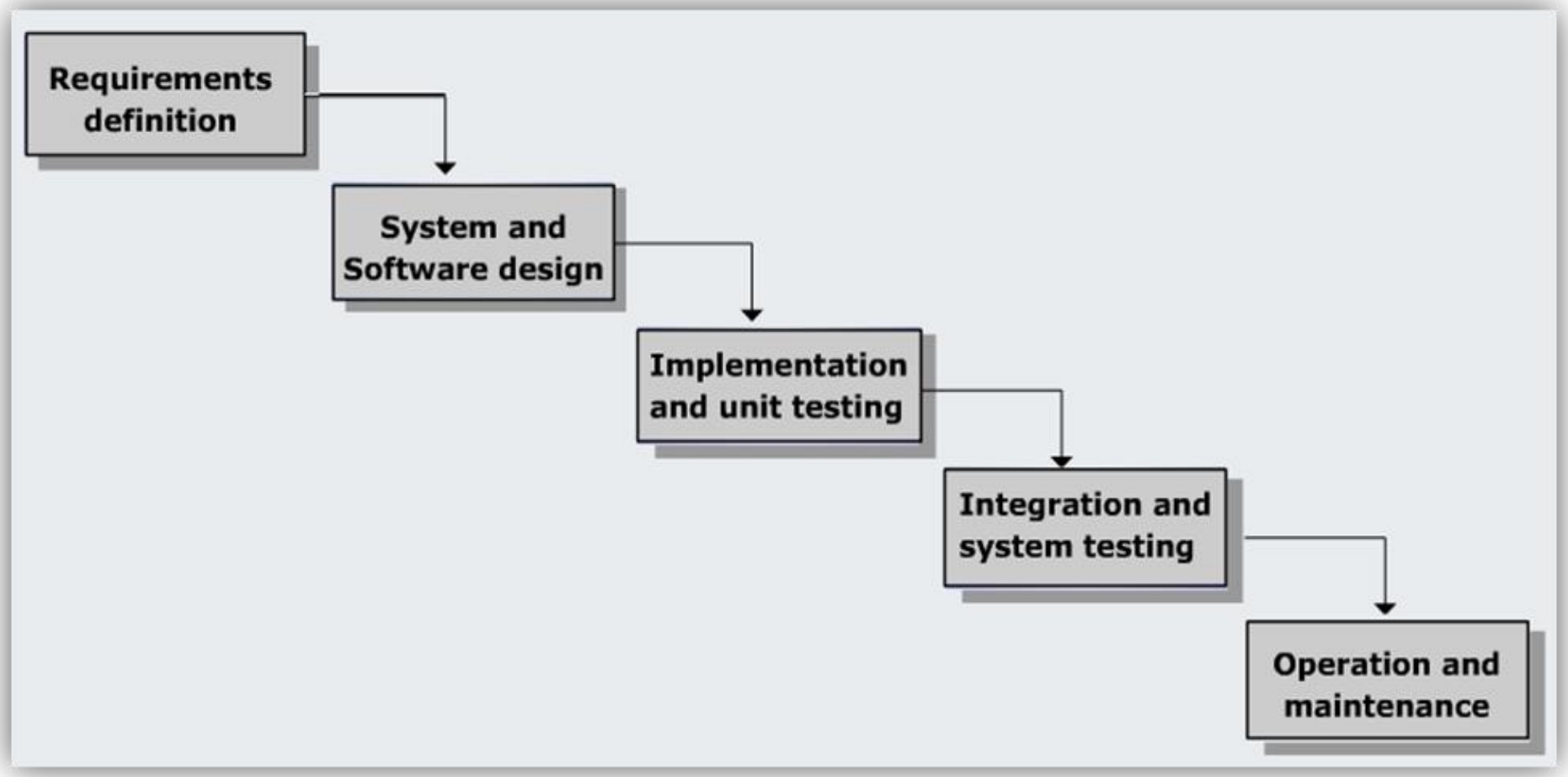
How to Choose Software Process models

- **A process model for Software Engineering is chosen based on the**
  - Nature of project and application
  - Methods and tools to be used
  - Controls and deliverables that are required

Waterfall Model

- Also called the classic life cycle or the linear sequential model
- It suggests a **systematic, sequential approach to software development** that **begins at the system leve**l and **progresses through analysis, design, coding, testing, and support.**
- It is known as waterfall because it has separate and distinct phases of specification and development. Once a phase is completed you can't repeat it.
- In order to be able to appreciate other life cycle models it is necessary to learn the classical waterfall model.

Waterfall Model



Phases in Waterfall Model

Requirements analysis and definition.

- The system's services, constraints, and goal are established and defined in a manner which is understandable both for users and development team.

System and software design.

- The system design process partitions the requirements to either hardware or software systems and establishes an overall system architecture.
- The software design involves representing software system functions in a form that can be translated into one or more executable programs.

Phases in Waterfall Model

Implementation and unit testing.

- During implementation phase, the software design is realized as a set of program units. Unit testing involves verifying that each unit meets its verification.

Integration and system testing.

- Individual units are integrated and tested to ensure that software requirements have been met.

Operation and maintenance.

- The system is installed and put into use. Maintenance involves correcting error, improving the system units and enhancing the system's services.

Pro's and Con's of Waterfall Model

Advantages:

- Simple to implement and manage.
- Enforces a disciplined software development approach

Drawbacks:

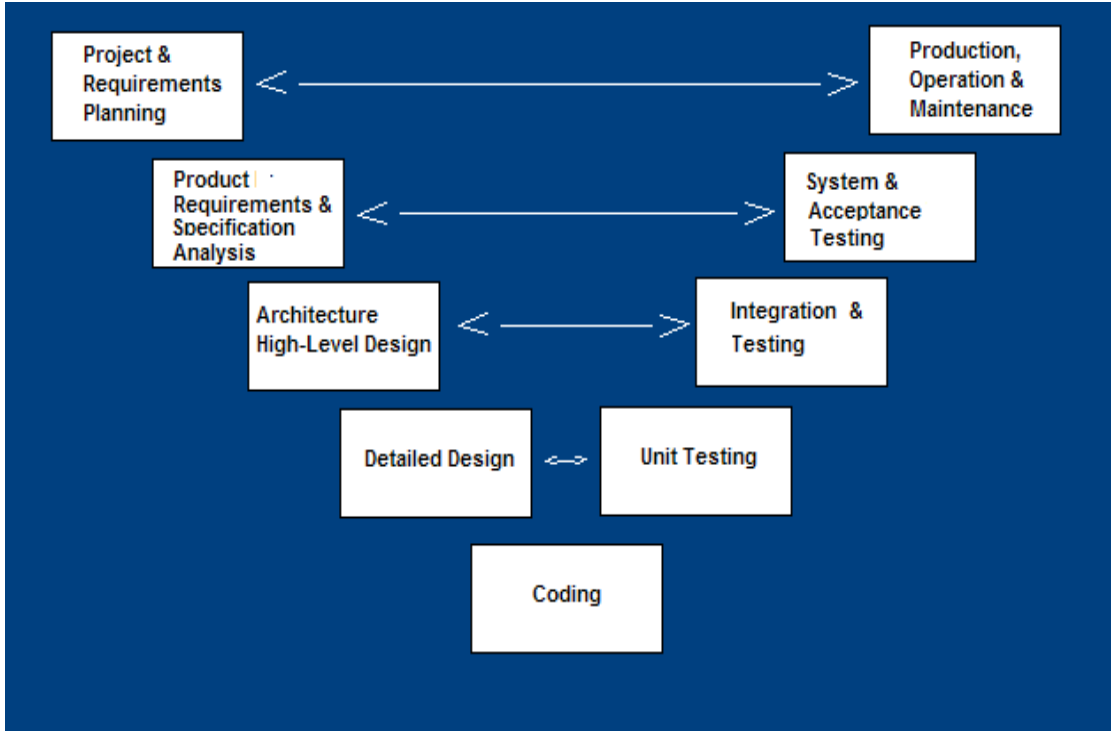
- Unable to accommodate changes at later stages
- A working version of the software is not available during development
- Unable to accommodate iteration
- Deadlock can occur due to delay of any step
- Model is not suitable for large projects

When to Use Waterfall Model

- Requirements are very well known
- Product definition is stable
- Technology is understood
- New version of an existing product
- Porting an existing product to a new platform.

V-Shaped SDLC Model

- A variant of the Waterfall that emphasizes the verification and validation of the product.
- Testing of the product is planned in parallel with a corresponding phase of development



V-Shaped SDLC Model Steps

- Project and Requirements Planning
  - allocate resources
- Product Requirements and Specification Analysis – complete specification of the software system
- Architecture or High-Level Design – defines how software functions fulfill the design
- Detailed Design – develop algorithms for each architectural component
- Production, operation and maintenance
  - provide for enhancement and corrections
- System and acceptance testing – check the entire software system in its environment
- Integration and Testing – check that modules interconnect correctly
- Unit testing – check that each module acts as expected
- Coding – transform algorithms into software

V-Shaped SDLC Model Strengths

- Emphasize planning for verification and validation of the product in early stages of product development
- Each deliverable must be testable
- Project management can track progress by milestones
- Easy to use

V-Shaped Weaknesses

- Does not easily handle concurrent events
- Does not handle iterations or phases
- Does not easily handle dynamic changes in requirements
- Does not contain risk analysis activities

When to use the V-Shaped Model

- Excellent choice for systems requiring high reliability – hospital patient control applications
- All requirements are known up-front
- When it can be modified to handle changing requirements beyond analysis phase
- Solution and technology are known

Structured Evolutionary Prototyping Model

- Developers build a prototype during the requirements phase
- Prototype is evaluated by end users
- Users give corrective feedback
- Developers further refine the prototype
- When the user is satisfied, the prototype code is brought up to the standards needed for a final product.

Structured Evolutionary Prototyping Steps

- A preliminary project plan is developed
- The model is source for a partial requirements specification
- A prototype is built with basic and critical attributes
- The designer builds
  - the database
  - user interface
  - algorithmic functions
- The designer demonstrates the prototype, the user evaluates for problems and suggests improvements.
- This loop continues until the user is satisfied

Structured Evolutionary Prototyping Strengths

- Customers can "see" the system requirements as they are being gathered
- Developers learn from customers
- A more accurate end product
- Unexpected requirements accommodated
- Allows for flexible design and development
- Steady, visible signs of progress produced
- Interaction with the prototype stimulates awareness of additional needed functionality

Structured Evolutionary Prototyping Weaknesses

- Tendency to abandon structured program development for "code-and-fix" development
- Bad reputation for "quick" methods
- Overall maintainability may be overlooked
- The customer may want the prototype delivered.
- Process may continue forever (scope creep)

When to use Structured Evolutionary Prototyping

- Requirements are unstable or have to be clarified
- As the requirements clarification stage of awaterfall model
- Develop user interfaces
- Short-lived demonstrations
- New, original development
- With the analysis and design portions of object- oriented development.

Rapid Application Model (RAD)

- Requirements planning phase (a workshop utilizing structured discussion of business problems)
- User description phase – automated tools capture information from users
- Construction phase – productivity tools, such as code generators, screen generators, etc. inside a time-box. ("Do until done")
- Cutover phase -- installation of the system, user acceptance testing and user training

RAD Strengths

- Reduced cycle time and improved productivity with fewer people means lower costs
- Time-box approach mitigates cost and schedule risk
- Customer involved throughout the complete cycle minimizes risk of not achieving customer satisfaction and business needs
- Focus moves from documentation to code.
- Uses modeling concepts to capture information about business, data, and processes.

RAD Weaknesses

- Accelerated development process must give quick responses to the user
- Risk of never achieving closure
- Hard to use with legacy systems
- Requires a system that can be modularized
- Developers and customers must be committed to rapid-fire activities in an abbreviated time frame.

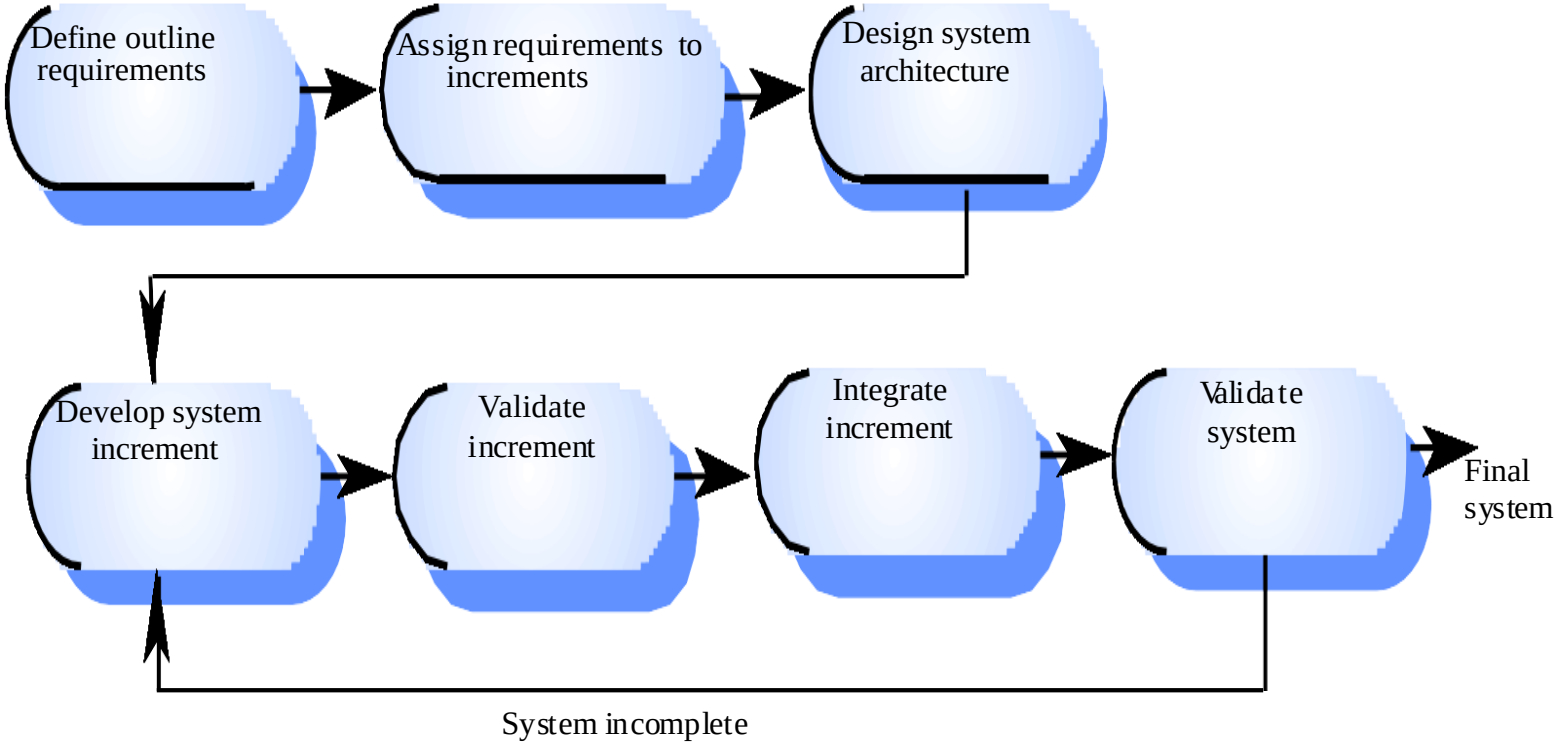
When to use RAD

- Reasonably well-known requirements
- User involved throughout the life cycle
- Project can be time-boxed
- Functionality delivered in increments
- High performance not required
- Low technical risks
- System can be modularized

Incremental Development

- Rather than deliver the system as a single delivery, **the development and delivery is broken down into increments** with each increment delivering part of the required functionality.
- **User requirements are prioritised** and **the highest priority requirements are included in early increments.**
- **Once the development of an increment is started, the requirements are frozen** though requirements for later increments can continue to evolve.

Incremental Development



Incremental Development –Advantages

- **Customer value** can be delivered with each increment so system functionality is available earlier.
- **Early increments** act as a prototype to help elicit requirements for later increments.
- Lower risk of overall project failure.
- **The highest priority system services** tend to receive the most testing.

Incremental Model Strengths

- Each release delivers an operational product
- Customer can respond to each build
- Uses “divide and conquer” breakdown of tasks
- Lowers initial delivery cost
- Initial product delivery is faster
- Customers get important functionality early
- Risk of changing requirements is reduced

When to use the Incremental Model

- Risk, funding, schedule, program complexity, or need for early realization of benefits.
- Most of the requirements are known up-front but are expected to evolve over time
- A need to get basic functionality to the market early
- On projects which have lengthy development schedules
- On a project with new technology

Spiral Development

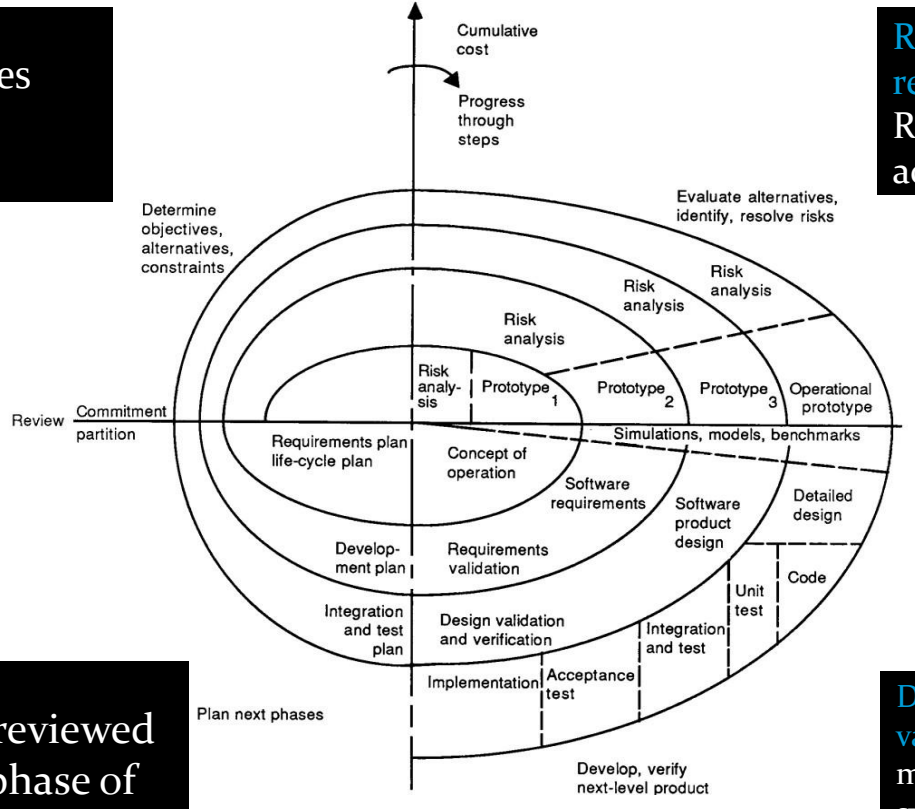
- **Process is conceived as a spiral** rather than as a sequence of activities with backtracking.
- **Each loop in the spiral represents a phase** in the process.
- **No fixed phases such as specification or design**

  - loops in the spiral are chosen depending on what is required.
- **Risks are explicitly assessed and resolved**

  throughout the process.

Spiral model (Boehm 87)

Objective setting
Specific objectives for the phase are identified.



Risk assessment and reduction
Risks are assessed and activities put in place to reduce the key risks.

Planning
The project is reviewed and the next phase of the spiral is planned.

Development and validation A development model for the system is chosen which can be any of the generic models.

Spiral model sectors

- **Objective setting**
  - Specific objectives for the phase are identified.
- **Risk assessment and reduction**
  - Risks are assessed and activities put in place to reduce the key risks.
- **Development and validation**
  - A development model for the system is chosen which can be any of the generic models.
- **Planning**
  - The project is reviewed and the next phase of the spiral is planned.

Spiral Model Strengths

- Provides early indication of insurmountable risks, without much cost
- Users see the system early because of rapid prototyping tools
- Critical high-risk functions are developed first
- The design does not have to be perfect
- Users can be closely tied to all lifecycle steps
- Early and frequent feedback from users
- Cumulative costs assessed frequently

Spiral Model Weaknesses

- Time spent for evaluating risks too large for small or low- risk projects
- Time spent planning, resetting objectives, doing risk analysis and prototyping may be excessive
- The model is complex
- Risk assessment expertise is required
- Developers must be reassigned during non-development phase activities
- May be hard to define objective, verifiable milestones that indicate readiness to proceed through the next iteration

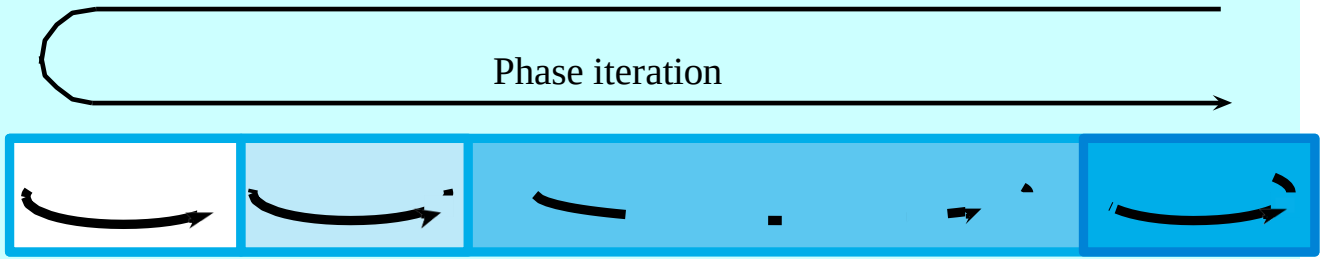
When to use Spiral Model

- When costs and risk evaluation is important
- For medium to high-risk projects
- Long-term project commitment unwise because of potential changes to economic priorities
- Users are unsure of their needs
- Requirements are complex
- New product line
- Significant changes are expected (research and exploration)

The (Rational) Unified Process

- A modern process model derived from the work on the UML.
- Normally described from 3 perspectives
  - A **dynamic** perspective that shows phases over time;
  - A **static** perspective that shows process activities;
  - A **practice** perspective that suggests good practice.

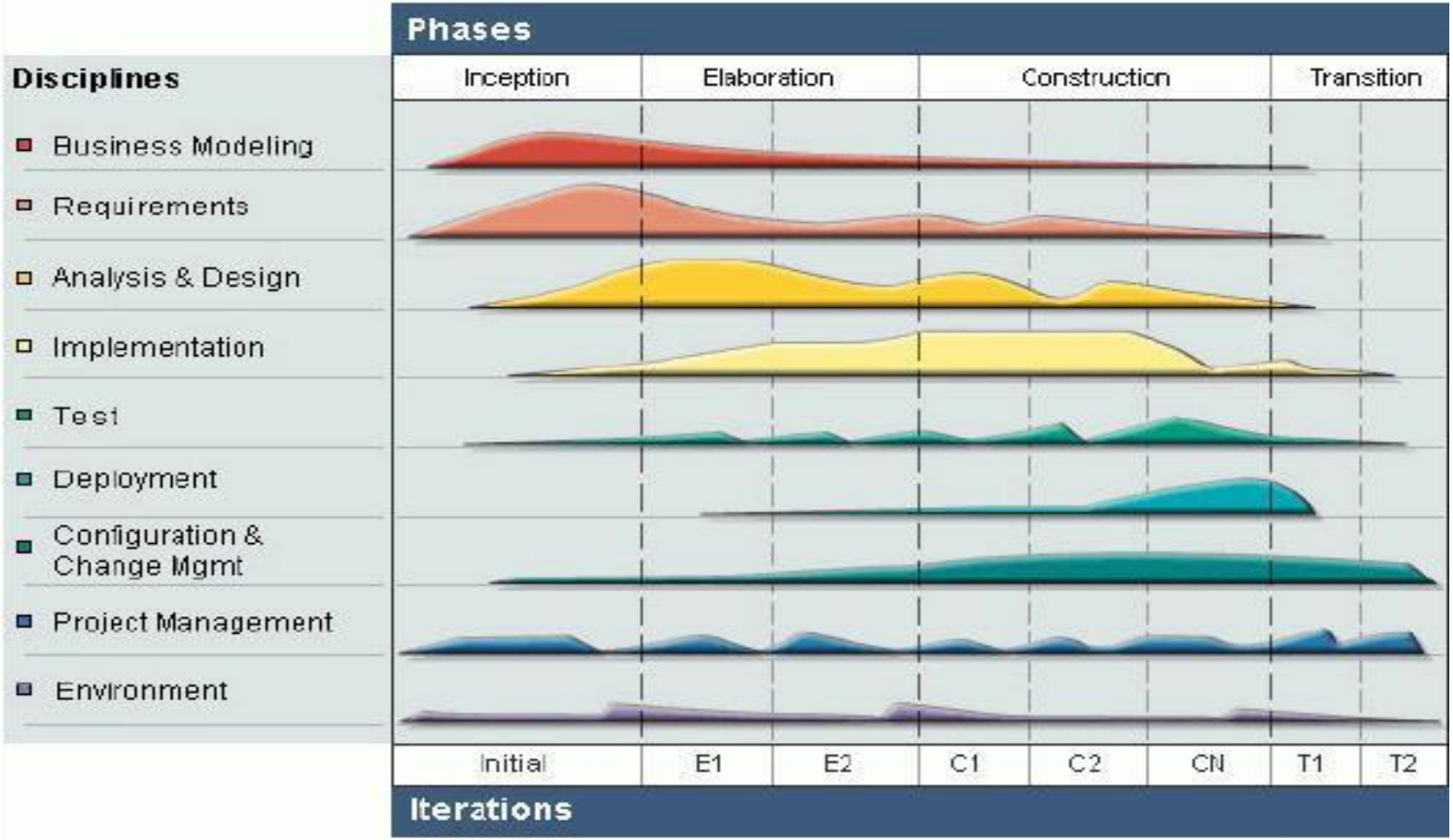
(R)UP phase model



(R)UP phases

- One cycle consists of four phases:
  - **Inception**
    - Establish the business case for the system.
  - **Elaboration**
    - Develop an understanding of the problem domain and the system architecture.
  - **Construction**
    - System design, programming and testing.
  - **Transition**
    - Deploy the system in its operating environment.

(R)UP phases and iterations

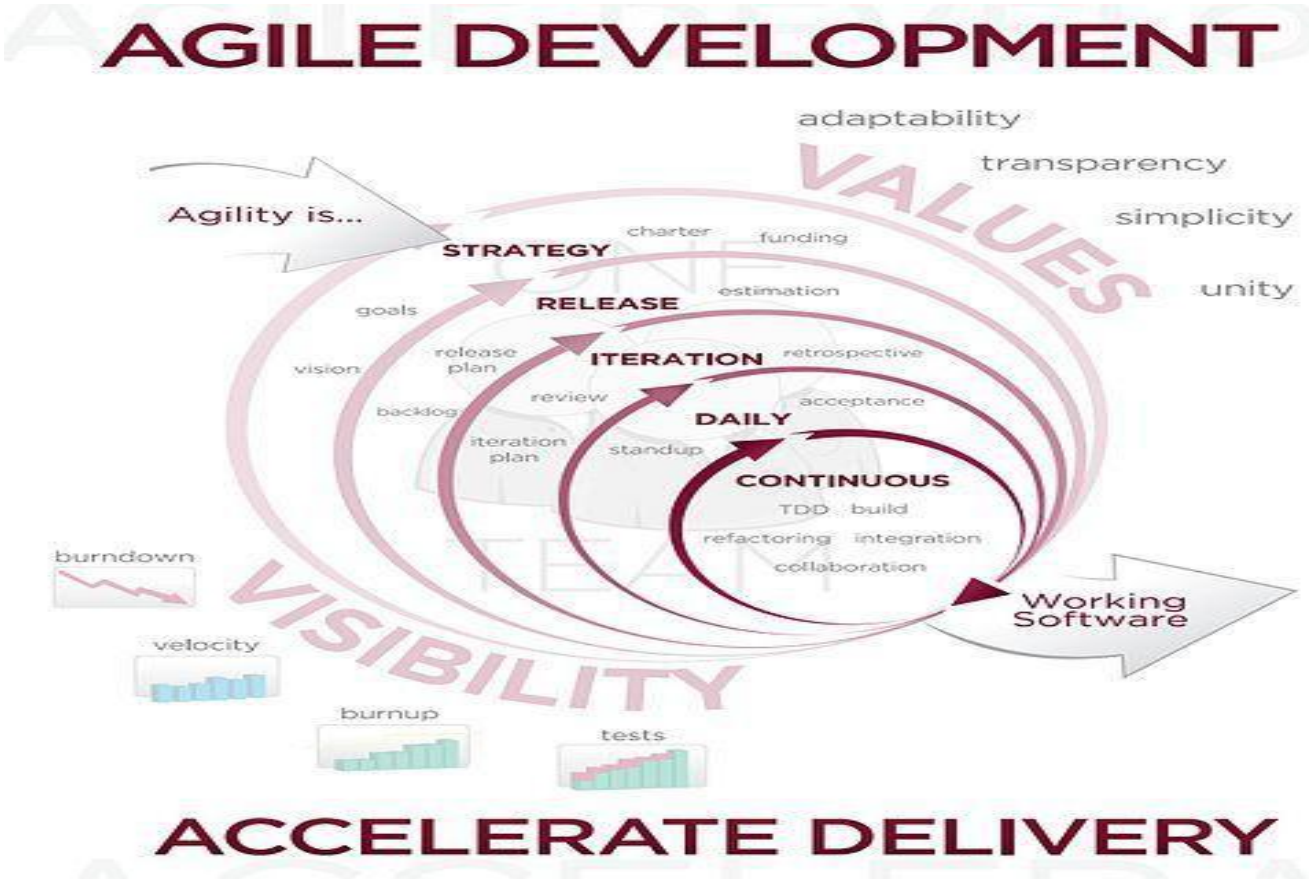


Picture taken from: http://www.ibm.com/developerworks/webservices/library/ws-soa-_term2/

(R)UP

- In each phase many different workflows (like management, environment, design, implementation workflow, etc.) can be addressed simultaneously.
- At the end of each cycle some kind of prototype or artifact are produced.
- The phases can be repeated many times (i.e. iterations), producing one or many prototypes or artifacts.
- During the cycles the requirements are stable which offers possibilities to plan the development process for this cycle.
- Between the cycles the requirements change.

Agile Development



What is an Agile method? (1)

- **Focus on the code rather than the design.**
- **Based on an iterative approach to software development.**
- **Intended to deliver working software quickly.**
- Evolve quickly to **meet changing requirements**.
- There are claims that agile methods are probably best suited to small/medium-sized business systems or PC products.

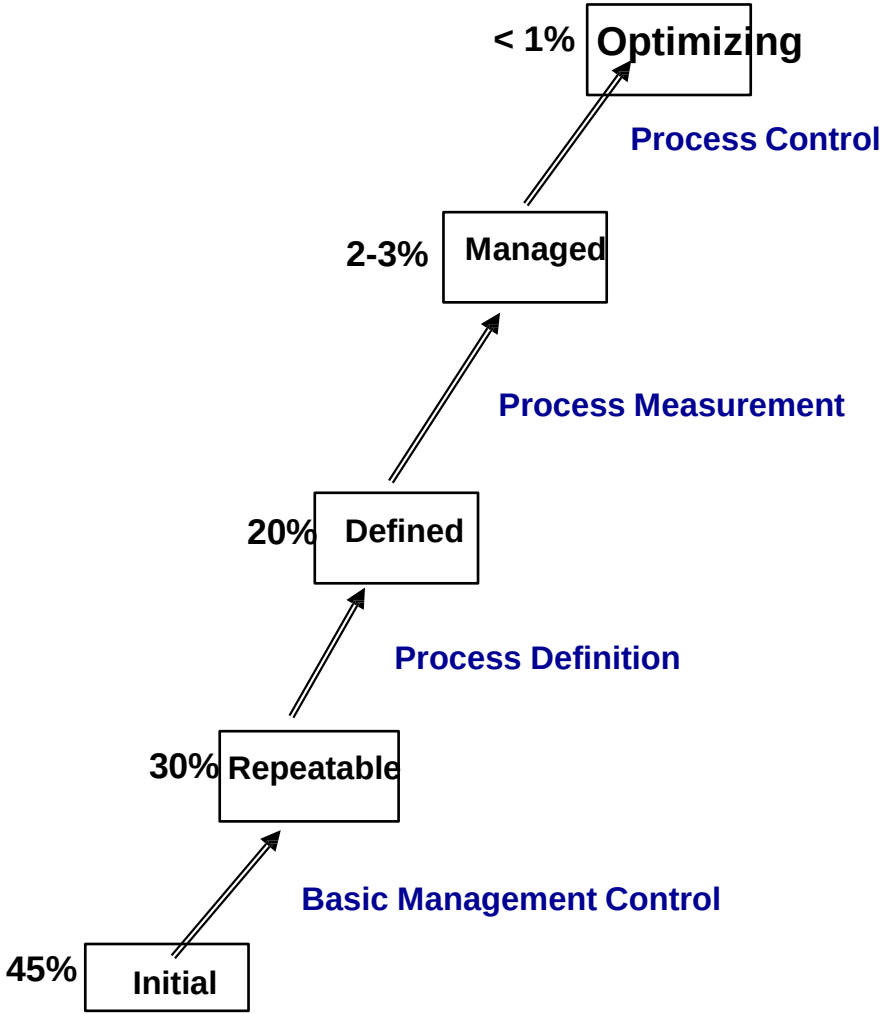
What is an agile method? (2)

- Agile methods are considered
  - **Lightweight**
  - **People-based** rather than Plan-based
- Several agile methods
  - No single agile method
  - Extreme Programming (XP) most popular
- Agile Manifesto closest to a definition
  - Set of principles
  - Developed by Agile Alliance

Software Process Improvement Efforts

- ***Carnegie Mellon University's Software Engineering Institute's Capability Maturity Model - (SEI's CMM)***
- ***International Standards Organization's 9001 Specification (ISO 9001)***
- ***Proprietary SPI's from consulting firms***

SEI Capability Maturity Model



CMM - Initial (Level 1)

- ***The software process is characterized as***

  ad hoc, occasionally even chaotic

- ***Few processes are defined***
- ***Success depends on individual effort and heroics***

"BASICALLY NO CONTROL"

CMM - Repeatable (Level 2)

- ***Basic project management processes are established to track cost, schedule, and functionality***
- ***The necessary process discipline is in place to repeat earlier successes on projects with similar applications***
- ***Success achieved through basic project management; not advanced technologies***

"BASIC MANAGEMENT CONTROL"

CMM - Defined (Level 3)

- ***The software process for both management and engineering activities is documented, standardized, and integrated into a standard software process for the organization***
- ***All projects use an approved, tailored version of the organization's standard software process for developing and maintaining software***
- ***Formality lends itself to improvement***

"PROCESS DEFINITION"

CMM - Managed (Level 4)

- Detailed measures of the software
- process and product quality are collected
- Both the software process and products are quantitatively understood and controlled
- A software metrics program is in use

"PROCESS MEASUREMENT"

CMM - Optimizing (Level 5)

- ***Continuous process improvement is enabled***

  by quantitative (metrics) feedback from the process

- ***Continuous process improvement is enabled by piloting innovative ideas and technologies***

"PROCESS CONTROL"

SW-CMM Process Assessment

- **General Classes of SW-CMM Appraisal**

1. Software Process Assessment
   - ➢ Determine state of organization's software process
2. Software Capability Evaluations
   - ➢ Identify contractors qualified to perform software work

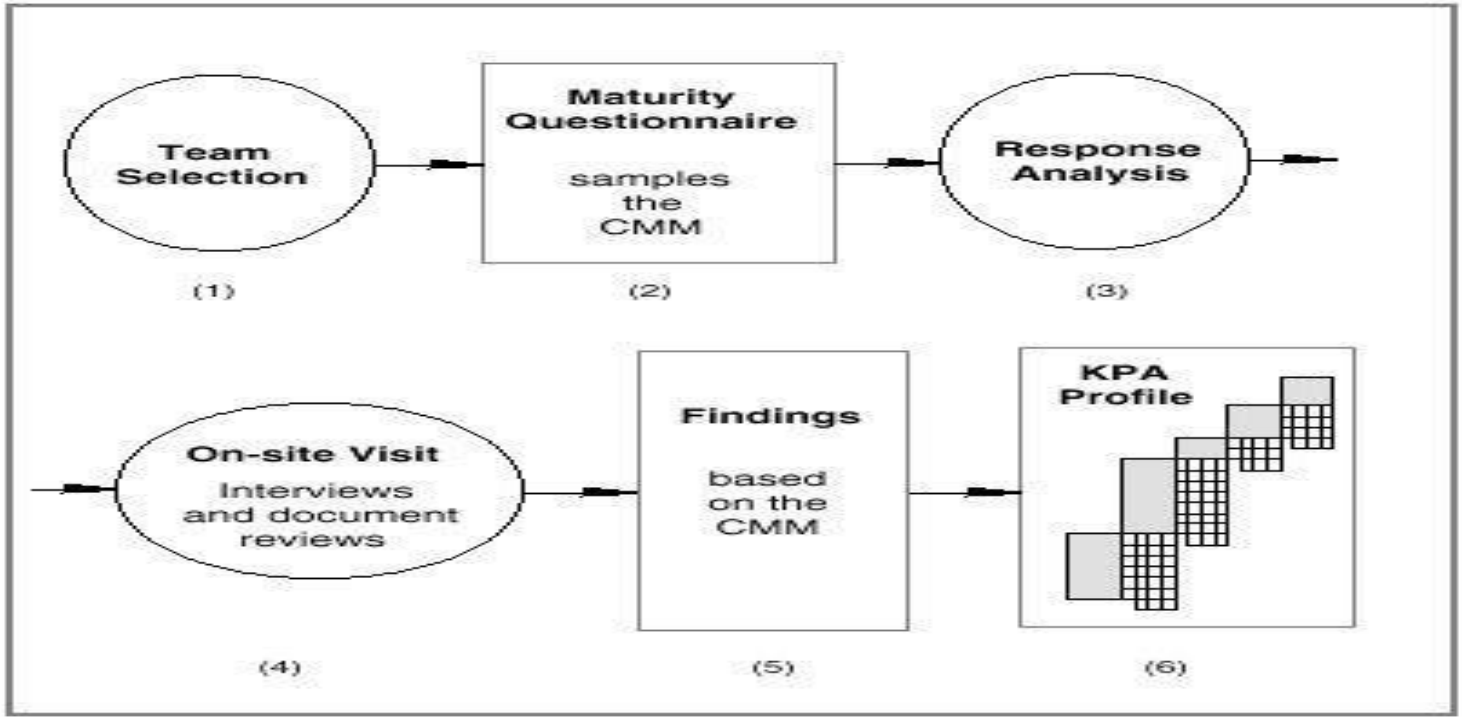
Software Process Assessment

- Software Process Assessment:
  - Identify improvement priorities within organization
  - Assessment team uses CMM to guide identifying & prioritizing findings
  - Findings & KPA guidance used to plan improvement strategy for organization

SW-CMM Process Assessment …

- Software Capability Evaluations:
  - Identify risks associated with a project or contract to build high quality on schedule & budget
  - During acquisition process, capability evaluation may be performed on bidders
  - Findings of an evaluation may be used to identify risk with using a contractor
  - Performed on existing contracts to monitor process performance

Software Process Assessment & Capability Evaluation Steps:



SW-CMM Process Assessment …

- Common Steps:
  - Team Selection
    - ➢ Select team trained in CMM
    - ➢ Knowledgeable in SE & mgmt
  - Maturity Questionnaire
    - ➢ Site reps complete questionnaire
  - Response Analysis
    - ➢ Analyze results of questionnaire
    - ➢ Investigation areas = KPAs

59

Conti…

- On-site Visit
  - Using results analysis, conduct on-site visit to view process areas
  - Using KPAs as guide, question, listen, review & synthesize info
  - Apply professional judgment
  - Document rationale for situations where KPAs not met

Conti…

- Findings
  - At end of on-site period, team produces list of findings
  - Identifies strengths & weaknesses of org's software processes
  - Software Process Assessment -> Basis for PI recommendations
  - Software Capability Evaluation -> Findings part of risk analysis
- KPA Profile
  - Team prepares KPA profile, showing where KPAs satisfied / not satisfied by organization
  - KPA can be satisfied and still have associated findings, as long as findings don't identify major problems achieving goals of KPA

Why Measure Software?

- *Metric* - quantitative measure of degree to which a system, component or process possesses a given attribute. "A handle or guess about a given attribute."
  - E.g., Number of errors found per person hours expended
- Determine the quality of the current product or process
- Predict qualities of a product/process
- Improve quality of a product/process

Motivation for Metrics

- Estimate the cost & schedule of future projects
- Evaluate the productivity impacts of new tools and techniques
- Establish productivity trends over time
- Improve software quality
- Forecast future staffing needs
- Anticipate and reduce future maintenance needs

Metrics in the Process Domain

- a metric as " a quantitative measure of the degree to which a system, component, or process possesses a given attribute".
- Process metrics are collected across all projects and over long periods of time
- They are used for making strategic decisions
- The intent is to provide a set of process indicators that lead to long- term software process improvement
- The only way to know how/where to improve any process is to
  - Measure specific attributes of the process
  - Develop a set of meaningful metrics based on these attributes
  - Use the metrics to provide indicators that will lead to a strategy for improvement

Conti…

- We measure the effectiveness of a process by deriving a set of metrics based on outcomes of the process such as
  - Errors uncovered before release of the software
  - Defects delivered to and reported by the end users
  - Work products delivered
  - Human effort expended
  - Calendar time expended
  - Conformance to the schedule
  - Time and effort to complete each generic activity

Conti…

Private metric

- There are "private and public" uses for different types of process data :
- Data private to the individual

  serve as an indicator for the individual only

Examples of metrics private to the individual

- defect rates (by individual)
- defect rates (by module)
- errors found during development

Conti…

Public metrics

- Public metrics assimilate information that originally was private to individuals and teams.
- Project-level defect
  - rates ,
  - effort,
  - calendar times,
  - related data
- are collected and evaluated in an attempt to uncover indicators that can improve organizational process performance.

Protocol of Process Metrics

- Use common sense and organizational sensitivity when interpreting metrics data
- Provide regular feedback to the individuals and teams who collect measures and metrics
- Don't use metrics to evaluate individuals
- Work with practitioners and teams to set clear goals and metrics that will be used to achieve them
- Never use metrics to threaten individuals or teams
- Metrics data that indicate a problem should not be considered "negative"
  - Such data are merely an indicator for process improvement
- Don't obsess on a single metric to the exclusion of other important metrics

END

Q?